Algorithms and an Exact Cross-Language Toolkit for VLSI Physical Design Automation

Wai-Shing Luk

***Abstract*—Physical design turns a gate-level netlist into a manufacturable layout, and every step of that flow—partitioning, placement, routing, clock-tree synthesis and timing closure—is an NP-hard combinatorial or geometric problem in disguise. This paper develops the algorithmic foundations of a complete, open physical-design flow and reports an exact, dependency-light implementation that is shared across Python, C++ and Rust. We first fix the domain rules that make the problem tractable: coordinates are arbitrary precision integers, geometry is rectilinear (Manhattan) unless stated otherwise, and algorithms are kept simple rather than merely easy. We then present, with proofs and complexity analysis, a rectilinear geometry toolkit (set-like generic operations, monotone polygons, a two-pass rectilinear convex hull, ray-casting point inclusion, convex decomposition, and Manhattan-arc merging segments); a primal-dual Steiner forest with a tight 2-approximation; a family of global routers with keepout avoidance and 3D extension; a prescribed-skew Deferred-Merge Embedding clock router with pluggable linear and Elmore delay models; and a static-timing, useful-skew and delay-padding closure loop. A recurring theme is the primal-dual method, which unifies graph covering and matching, and Steiner forest routing. We validate the implementation on synthetic grids and standard benchmarks, verify cross-language agreement, and document a silent failure mode that a research-grade toolkit must expose: asymptotically quadratic hot paths hidden behind a correct answer.**

I. Introduction

A modern integrated circuit is a physical object with billions of components, and the process that turns an abstract netlist into the polygons that a foundry can fabricate is *physical design automation*. The field sits at the intersection of computer science and electrical engineering: the objects are geometric, the objectives are electrical, and the problems are combinatorial. The classical pipeline partitions the netlist, places the cells, routes the wires, distributes the clock, and closes timing [1], [2], [3]; each stage is a large-scale optimization problem, and most of them are NP-hard.

Three properties of the domain shape every algorithm in this paper. First, layout coordinates are integers. A manufacturing grid may be a million units across, and floating-point rounding is not merely imprecise but illegal: two edges that should be collinear may miss each other, and a design-rule check fails. We therefore use arbitrary-precision integers, accepting the obligation to reason about overflow in C++ and Rust where the arithmetic is fixed-width. Second, geometry is *rectilinear*: unless a 45-degree abstraction is stated explicitly, every edge is axis-parallel. This restriction simplifies design rule checking, area computation and overlap tests, and it is the reason that Manhattan distance, not Euclidean distance, is the natural metric. Third, the scale is extreme—thousands of millions of objects—but the objects are individually small: a polygon has at most a hundred vertices, a net a handful of pins, and a chip a score of metal layers. An algorithm that is simple in the sense of having few moving parts, even if not easy to discover, is worth more than a sophisticated one that is hard to verify.

This paper develops the algorithmic foundations of such a flow and reports an exact, dependency-light implementation shared across Python, C++ and Rust. Our contributions are as follows.

1. We fix a domain-specific calculus of rectilinear geometry: generic set-like operations dispatched by structural typing, an interval/point algebra in which a rectangle is a point of intervals and a segment is a half-interval point, monotone polygon construction, a two-pass rectilinear convex hull, ray-casting point inclusion, and convex decomposition (Section III).
2. We present graph and hypergraph covering, matching and independent-set algorithms built on the primal-dual schema, and the netlist abstraction they serve (Section IV).
3. We state the Steiner forest problem, derive the primal-dual algorithm from the undirected cut relaxation, prove its tight 2-approximation, and give an efficient union-find implementation (Section V).
4. We describe a global router with three strategies, keepout avoidance and a 3D extension, and contrast the state of the art in FPGA routing (Section VI).
5. We give the Deferred-Merge Embedding algorithm for prescribed-skew clock trees with pluggable linear and Elmore delay models, and the static-timing, useful-skew and delay-padding loop that closes timing (Section VII).
6. We validate the implementation on synthetic grids and standard benchmarks, verify agreement across the three language implementations, and document two silent-failure modes—balance-constraint violations and quadratic hot paths—that a trustworthy tool must expose (Section VIII).

II. Domain-Specific Preliminaries

A. Exact Integer Geometry

All coordinates are integers drawn from an arbitrary-precision domain (Python's `int`, C++'s fixed-width `int64_t` with explicit overflow care, Rust's `i64`/`i128`). Let d be the Manhattan, or L_1, distance between two points $x = (x_1, x_2)$ and $y = (y_1, y_2)$:

$$d(x, y) = |x_1 - y_1| + |x_2 - y_2|. \tag{1}$$

Whereas the Euclidean metric makes a point's unit ball a circle, the L_1 unit ball is a diamond, and the L_∞ unit ball is an axis-aligned square. This duality is exploited throughout: the rectilinear Voronoi diagram is an L_1 diagram computed by an L_∞ plane sweep on the transformed coordinates $u = x + y$, $v = x - y$.

B. The Rectilinear Assumption and the Scale of the Problem

We adopt the following rules, which recur in every later section.

- **Integer coordinates.** Never floating point. Watch for overflow in addition, subtraction and multiplication.
- **Rectilinear geometry.** Vertical and horizontal edges, plus 45-degree abstract lines when the model calls for them.
- **Scale.** The number of physical objects is on the order of 10^9, but the exceptions are small and exploitable: polygons have at most about 100 vertices, nets a handful of pins (apart from high-fanout nets), metal layers at most 20, and keepouts at most 10.
- **Simplicity.** An algorithm should be simple, which is not the same as easy.
- **Ownership.** In C++, no virtual functions unless needed, and `unique_ptr` in preference to `shared_ptr`, because a shared pointer hides ownership cycles and pays an atomic reference count.

These rules are not stylistic preferences; they are the reason the algorithms below are exact, fast and portable across languages.

III. Rectilinear Geometry

A. Generic Set-Like Operations

The primitives of the toolkit are *set-like*: two objects may overlap, one may contain another, they may intersect, and they have a minimum distance. We want these operations to work on a scalar, an interval, a point, a rectangle or a segment without duplicating code. The trick is structural dispatch: an operation is implemented by whichever operand has the corresponding member, and otherwise the operands are treated as scalars.

```
def overlap(lhs, rhs) -> bool:
    if hasattr(lhs, "overlaps"):
        return lhs.overlaps(rhs)
    elif hasattr(rhs, "overlaps"):
        return rhs.overlaps(lhs)
    else:  # assume scalar
        return lhs == rhs
```

The same template defines `contain`, `intersection`, `hull`, `enlarge`, `nearest`, `blocks` and `measure_of`. The C++20 implementation uses a `requires` expression for compile-time dispatch and the Rust implementation a trait bound, so that the operation is resolved without virtual dispatch in all three languages:

```
template <typename U1, typename U2>
constexpr auto overlap(const U1 &lhs, const U2 &rhs) -> bo
    if constexpr (requires { lhs.overlaps(rhs); }) {
        return lhs.overlaps(rhs);
    } else if constexpr (requires { rhs.overlaps(lhs); })
        return rhs.overlaps(lhs);
    } else /* constexpr */ {
        return lhs == rhs;
    }
}
```

For intervals, `overlaps(other)` returns `not (self < other or other < self)`, where the ordering `$<$` is interval-disjointness: $I < J$ iff $\mathrm{ub}(I) < \mathrm{lb}(J)$. Two intervals overlap iff neither lies strictly to the left of the other. A point overlaps another iff both coordinates do, and a point *blocks* another when one coordinate is contained and the other contains, i.e. the two touch without either containing the other—a relation used to detect a wire grazing an obstacle without entering it.

B. Intervals, Points and the Generic-Shape Algebra

An `Interval` has a lower bound `lb` and an upper bound `ub`, with `length()`, `contains()`, `intersect_with()` and `min_dist_with()`. A `Point` has coordinates `xcoord` and `ycoord` and a `displace()` that returns a new point translated by a vector; a `Vector2` adds a `cross` product. Because the coordinate types are themselves generic, one class covers the whole family of axis-parallel shapes of Table I.

The `hull` operation returns the bounding box of two objects: for scalars it builds the interval that spans them, and for points or intervals it applies `hull` coordinate-wise. The `enlarge` operation grows a scalar into the interval of half-width `rhs` and grows a point into a rectangle. Together they give the bounding-box and offsetting primitives on which legalization, keepout construction and buffer insertion rest.

C. Rectilinear Polygons

A *rectilinear polygon* is a polygon whose edges alternate between horizontal and vertical, so that every interior angle is 90° or 270°. We represent it by an origin plus a list of edge vectors relative to the origin,

$$P = \big(o,\ [v_0, v_1, \ldots, v_{n-1}]\big),$$

where o is a reference point and each v_i is the displacement from one vertex to the next. Translation is then an $O(1)$

Table I: Rectilinear shapes as nested generic points.

Shape	Representation
Point	`Point<int, int>`
Rectangle	`Point<Interval, Interval>`
Horizontal seg.	`Point<Interval, int>`
Vertical seg.	`Point<int, Interval>`
Box (3D)	`Point<Point<int,int>, int>`

Algorithm 1 Rectilinear convex hull by two monotone passes

Require: vertices S in cyclic order, orientation flag σ
1: $S \leftarrow \textsc{MakeXMonotoneHull}(S, \sigma)$
2: $S \leftarrow \textsc{MakeYMonotoneHull}(S, \sigma)$
3: **return** S

update of the origin (`__iadd__`/`__isub__` on the origin only), which is what makes incremental placement cheap: moving a macro does not touch its vertex list.

Two classical predicates are needed repeatedly. The *signed area* follows from the shoelace formula,

$$A = \frac{1}{2} \sum_{i=0}^{n-1} \big(x_i y_{i+1} - x_{i+1} y_i \big), \tag{2}$$

whose sign gives the orientation. Rather than accumulate the shoelace terms directly, the implementation sums the cross-products of consecutive relative vectors, which is numerically identical on integer coordinates but avoids re-materialising absolute vertices. *Point inclusion* uses ray casting: a horizontal ray cast in the $+x$ direction is toggled at each edge that straddles the query ordinate,

$$\text{inside}(q) \iff \#\{\text{toggles}\} \text{ is odd}, \tag{3}$$

with the half-open convention $\text{lb} \le q_y < \text{ub}$ on each edge so that a ray through a vertex is counted once, not twice or zero times.

D. Monotone Polygons and the Rectilinear Convex Hull

A polygon is *x-monotone* if every vertical line meets it in at most two points, and *y-monotone* if every horizontal line does. A rectilinear polygon is convex in the rectilinear sense exactly when it is both x-monotone and y-monotone. This characterization yields an $O(n)$ convex hull by two passes (Fig. 1, then Fig. 2). Given the vertex list and an orientation flag, the first pass constructs the x-monotone hull: locate the leftmost and rightmost vertices, partition the remaining vertices into an upper and a lower chain, sort each chain along the sweep direction, and concatenate lower then upper. The second pass applies the same procedure along y. Each pass walks a circular doubly linked list and removes a vertex whenever the turn at that vertex is concave, so the total work is linear.

E. Convex Decomposition

Many geometric algorithms are stated for convex shapes. A rectilinear *convex decomposition* splits a concave polygon into non-overlapping convex rectilinear pieces by recursive cutting. At each step the algorithm finds a concave vertex—one where the two incident edges turn in opposite coordinate senses—and cuts it to the nearest vertex that is visible along an axis-parallel line, adding one segment and splitting the polygon into two. Because every cut strictly reduces the number of concave vertices, the recursion terminates; the worst-case cost is $O(n^2)$ and the space is $O(n)$. The cut primitive is exactly the operation used by global routers to negotiate obstacles, which is why the decomposition lives next to the router in the same package (Fig. 3).

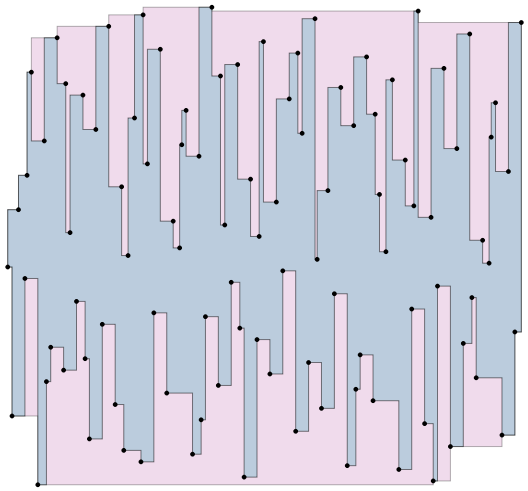


Figure 1: The x-monotone hull stage, before the y-monotone pass completes the rectilinear convex hull.

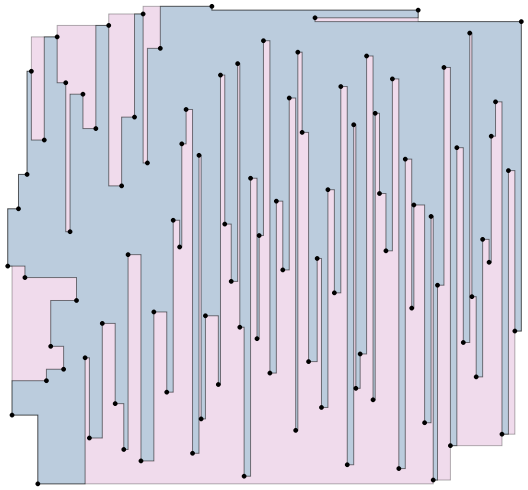


Figure 2: The rectilinear convex hull of a point set, obtained by the x- then y-monotone passes.

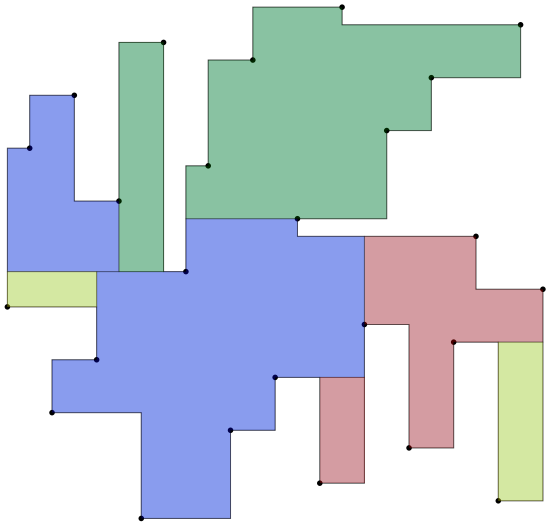


Figure 3: Convex decomposition of a concave rectilinear polygon into non-overlapping convex pieces (colored); vertices are marked.

F. Manhattan Arcs and Merging Segments

The last geometric primitive is the 45-degree *merging segment* used by clock-tree synthesis. In the L_1 metric the set of points at distance at most r from a point is a diamond, and in the transformed coordinates $u = x + y$, $v = x - y$ it is an axis-aligned square; the diamond is therefore the L_1 analogue of the circle, and a maximal set of optimal merge points is the L_1 analogue of a disc. A Manhattan arc stores one such (possibly degenerate) region as a pair of intervals and supports `min_dist_with`, `merge_with(other, extend_left)` and `nearest_point_to`. These are the operations required by the Deferred-Merge Embedding algorithm of Section VII; Fig. 4 contrasts them with their Euclidean counterparts.

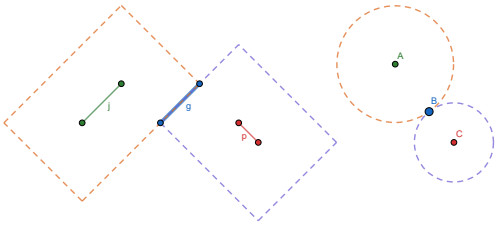


Figure 4: The L_1 merging-segment operations: intersecting two enlarged Manhattan arcs (left) is the analogue of intersecting two circles (right) in a Euclidean Voronoi construction.

G. Rectilinear Voronoi Diagrams

The rectilinear Voronoi diagram assigns to each site the region of points closer to it than to any other site *in the L_1 metric*, and it is the geometric engine behind buffer insertion, wire-width estimation and nearest-obstacle queries. It is computed by the divide-and-conquer algorithm of Lee and Wong [4], which recursively splits the sites by x, then merges the left and right diagrams by walking the *merge line*—the bisector of the closest cross pair—upward and downward, trimming the bisectors it crosses. The rotation $u = x + y$, $v = x - y$ turns the L_1 metric into the L_∞ metric,

$$|\Delta x| + |\Delta y| = \max\big(|\Delta u|, |\Delta v|\big), \tag{4}$$

which is why an L_1 diagram can be computed by an L_∞ sweep on the transformed plane and why the L_1 unit ball is a square.

The difficulty is degeneracy. The L_1 bisector of two sites,

$$|x - x_1| + |y - y_1| = |x - x_2| + |y - y_2|, \tag{5}$$

is a line when one coordinate difference dominates, but a whole *region* when $|\Delta x| = |\Delta y|$: the two sites lie on a diagonal, and every point of a quadrant is equidistant. The naive fix—a numerical *nudge*, $d_0 \mathrel{+}= 10^{-10} d_1$, $d_1 \mathrel{+}= 2 \times 10^{-10} d_0$ breaks the integer coordinates on which the rest of the toolkit insists, is directionally arbitrary, is an $O(N^2)$ pairwise scan, and leaks floats into the output. We replace it with an exact, deterministic disambiguation: when $|\Delta x| = |\Delta y|$ the bisector orientation is fixed by the sign of the diagonal, a *three-way* comparison distinguishes parallel from collinear segments, and a collinear overlap returns its lowest-y endpoint. Because the tie-break is applied in every predicate that can observe the degeneracy—bisector construction, orientation, overlap test, and the trim sort—the resulting cells neither overlap nor leave gaps. The implementation passes 58 unit tests and a 50-seed randomized stress test, and it is verified to place the three concurrency bisectors of the worked example at the exact L_1 circumcenter $(-4, -1)$ with equal distance 5. A systematic study of four strategies (canonical bisector, minimal symbolic perturbation, exact arithmetic, post-processing merge) over a 42-case corpus shows that only exact arithmetic with a region-aware merge eliminates the residual violations [5], which is the design the toolkit adopts.

IV. Netlist and Graph Algorithms

A. The Primal-Dual Schema

Many physical-design subproblems are covering or matching problems on graphs and hypergraphs, and most are NP-hard. The primal-dual method is a single algorithmic idea that handles them all: instead of solving the integer program, one raises the dual variables *uniformly* over the currently violated structures until some constraint becomes tight, then fixes the corresponding primal decision and repeats. The result is a feasible primal solution whose cost is at most a constant times the total dual value, i.e. a constant-factor approximation. Kept as a library, this schema solves minimum vertex cover, minimum cycle cover and minimum odd-cycle cover in a graph or hypergraph,

minimum maximal independent set, and minimum maximal matching, all from the same driver parameterized by a feasibility predicate.

A representative case is the *minimum-weight maximal independent set*. An independent set has no edge between two of its members, maximality forbids adding another, and minimality asks for the least total weight. The primal-dual driver grows dual variables on edges until one tightens, adds the corresponding vertex, and removes its closed neighbourhood; the resulting set is independent, maximal, and within a logarithmic factor of optimal. Minimum maximal *matching*—a disjoint set of hyperedges that cannot be extended—is the dual construction used for graph coarsening.

B. Netlists as Graphs

A netlist is the natural input to every stage, and it is represented as a hypergraph: a list of modules, a list of nets, and a list of pins connecting them. The abstraction exposes only what the algorithms need—the number of modules, nets and pins; the maximum degree; and net and module weights—and provides readers for the two interchange formats used in practice, the hMetis hypergraph format and Yosys synthesis JSON. In the Yosys reader, cells become weight-one modules and I/O ports become fixed weight-zero pads, so that the partition or placement respects the package boundary. Keeping the graph algorithms independent of the netlist representation—operating on a `NetworkX` graph or a plain adjacency structure—lets the same covering, matching and independent-set routines serve routing and netlist processing.

V. Steiner Forest via Union-Find and Primal-Dual

A. Problem and Formulation

Global routing must connect many nets, and the wirelength-optimal model for a single multi-terminal net is the rectilinear Steiner *tree*; for many nets that may share wires, the model is the Steiner *forest*. Given an undirected graph $G = (V, E)$ with non-negative edge weights $w_e \ge 0$ and k terminal pairs (s_i, t_i), the Steiner forest problem asks for a minimum-cost edge set $F \subseteq E$ such that every pair is connected in (V, F):

$$\min_{F} \sum_{e \in E} w_e x_e \quad \text{s.t.} \quad \sum_{e \in \delta(S)} x_e \ge 1 \;\; \forall S \in \mathcal{S}_i, \;\; x_e \in \{0,1\}, \tag{6}$$

where $\mathcal{S}_i$ is the family of vertex subsets that separate s_i from t_i and $\delta(S)$ is the cut of S. The problem is a generalization of Steiner tree and is NP-hard and APX-hard (Fig. 5). Its LP relaxation—the undirected cut relaxation—has the dual

$$\max_{y \ge 0} \sum_{S} y_S \quad \text{s.t.} \quad \sum_{S:\, e \in \delta(S)} y_S \le w_e \;\; \forall e \in E, \tag{7}$$

where y_S is the width of a *moat* surrounding S: no edge may be crossed by moats of total width exceeding its cost.

Algorithm 2 Primal-dual Steiner forest with union-find

Require: graph $G = (V, E)$, weights w, terminal pairs $\{(s_i, t_i)\}$
$y \leftarrow 0$; $F \leftarrow \emptyset$; $\text{paid}(e) \leftarrow 0$
while some pair is unconnected **do**
  find the active components $\mathcal{A}$
  $\Delta \leftarrow \min_{e \text{ crossing}} (w_e - \text{paid}(e)) / \text{num_active}(e)$
  $\text{paid}(e) \leftarrow \text{paid}(e) + \Delta \cdot \text{num_active}(e)$ for every crossing e
  add every tight e to F and union its endpoints
end while
reverse-delete: remove $e \in F$ if $F \setminus \{e\}$ still connects every pair
return F

Complementary slackness says that an edge enters the solution only when this constraint is tight.

B. The Primal-Dual Steiner Forest

The algorithm of Agrawal, Klein and Ravi [6] builds the primal solution and the dual simultaneously. A connected component of the current partial solution is *active* if it contains a terminal still separated from its partner. In each iteration all active components raise their dual variables uniformly until some edge becomes tight; that edge is added, merging two components, and the process repeats until every pair is connected. The uniform increase is limited by the first tight edge; if c_e is its cost and $\text{paid}(e)$ is the moat width already crossing it, then

$$\Delta_e = \frac{c_e - \text{paid}(e)}{\text{num_active}(e)}, \tag{8}$$

and the global step is the minimum of Δ_e over crossing edges. Edges are stored in a union-find structure, so a component query costs $O(\alpha(n))$ amortized—effectively constant. Algorithm 2 collects the procedure.

After the main loop, a *reverse-delete* pass removes redundant edges in reverse order of addition: an edge that lies on a cycle can be dropped without disconnecting any pair. This pruning is not merely cosmetic; it is what makes the approximation proof go through.

C. Why the Ratio Is Two

The algorithm is a 2-approximation: the returned forest F' satisfies $c(F') \le 2 \cdot \text{OPT}$. The dual value is a lower bound, $\sum_S y_S \le \text{OPT}$, and because every edge of F' was tight when added, the cost of the forest can be written through the dual variables as

$$c(F') = \sum_{S} y_S \, |F' \cap \delta(S)|. \tag{9}$$

The proof reduces to a single lemma: for the active components A_t of any iteration, $\sum_{C \in A_t} |F' \cap \delta(C)| \le 2|A_t|$. The reverse-delete step ensures that $F'[A_t]$ is a *forest*, and in a forest the sum of the degrees of the leaves is at most twice the number of vertices. Summing the per-iteration bound telescopes to

$$c(F') = \sum_{t} \Delta_t \sum_{C \in A_t} |F' \cap \delta(C)| \le 2 \sum_{t} \Delta_t |A_t| = 2 \sum_{S} y_S \le 2 \cdot \text{OPT}\,. \tag{10}$$

The factor 2 is tight for the cut relaxation and has stood as a barrier for decades; a recent deterministic algorithm [7] achieves $2-10^{-11}$, breaking it. In practice the method is far better than its worst case: on the SteinForestLib benchmark the worst observed ratio is 1.269, and the algorithm is one to three orders of magnitude faster than greedy or randomized alternatives, with a running time that does not grow with the number of terminal pairs. Table II reports the observed quality.

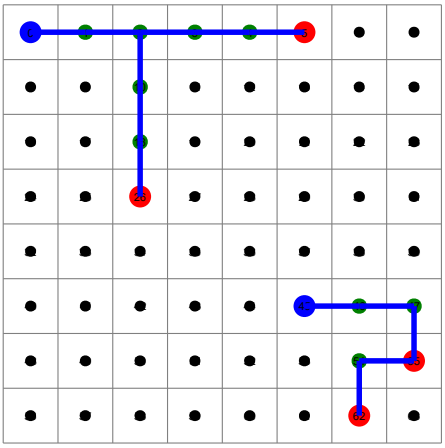


Figure 5: A Steiner forest on an 8×8 grid: each terminal pair is joined by a tree, with Steiner nodes where paths merge.

VI. Global and FPGA Routing

A. Global Routing with Keepouts and 3D Extension

Global routing assigns each net a coarse path through routing regions; detailed routing then pins it to exact tracks. The toolkit's global router holds a source, a list of terminals, a list of *keepouts* (rectangular obstacles such as pre-routed wires, macros and IP blocks), and builds a *routing tree* whose nodes are of three kinds: source, Steiner point, and terminal. It exposes three strategies of increasing cost and decreasing wirelength:

1. `route_simple` connects every terminal directly to the nearest node of the existing tree, in $O(nm)$ time for n terminals and m tree nodes.
2. `route_with_steiners` inserts Steiner points so that nets share paths, minimizing total wirelength in $O(nmk)$ time with k keepouts.
3. `route_with_constraints` is performance-driven: it caps every terminal's length by a fraction of the worst connection,

$$\text{allowed length} = \alpha \times \text{worst wirelength}, \qquad \alpha \in [0.5, 2.0], \tag{11}$$

and falls back to the full budget when the constraint cannot be met.

A keepout is a rectangle represented as a point of intervals, `Point(Interval, Interval)`, and a path is blocked when the rectangle contains the nearest point or the path `blocks` the rectangle edge. Because the `Point` class is generic, the same router runs in three dimensions by replacing the planar point with a nested point whose second coordinate is the metal layer, $d = |\Delta x| + |\Delta y| + |\Delta z|$; allowing a net to change layers lets a route of planar length L be embedded in roughly $L/2 + z$ of wire, about a 50% saving when the same-layer route would cost $L_2 + 2z$. The router is deliberately a *steiner-point-guided* global router: its output is a tree that the detailed router refines and a congestion map that the placer can exploit; Fig. 6 and Fig. 7 show the planner in two and three dimensions, and Fig. 8 the map renderer.

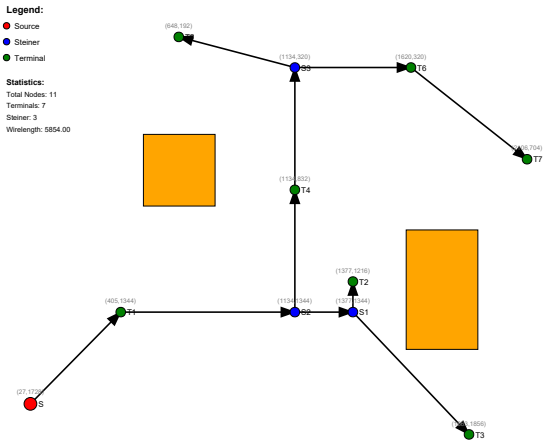


Figure 6: Global routing with Steiner-point insertion and rectangular keepouts (orange): the tree avoids the obstacles and shares segments between terminals.

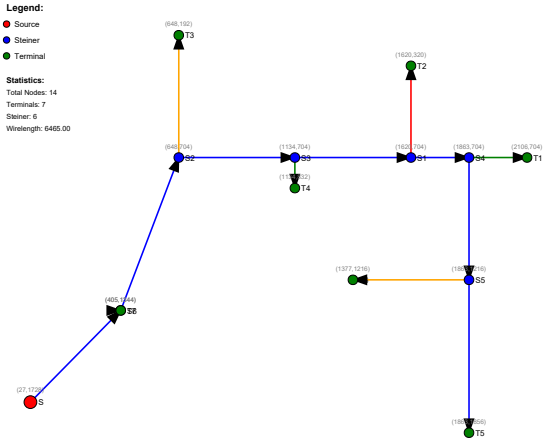


Figure 7: The same router in three dimensions: terminals on different metal layers are connected through the layer coordinate z, at the cost of vias.

B. FPGA Routing: Maze, Pathfinder, and Beyond

FPGA routing differs from ASIC routing in that the resources are *fixed* and the architecture is *explicit*: an island-style fabric of logic blocks, horizontal and vertical

Table II: Primal-dual Steiner forest versus alternatives (SteinForestLib).

algorithm	theoretical ratio	worst observed	runtime
PD-SF (this work)	2	1.269	fastest
PG-SF (greedy)	$\Omega(\log n)$	1.115	moderate
TM-SF (randomized)	$O(\log n)$	7.086	slow

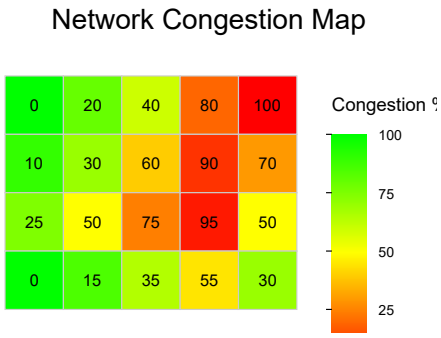


Figure 8: The congestion-map renderer: a caller-supplied per-region utilization grid drawn as a green-to-red heat map (schematic).

channels, connection boxes of flexibility F_c, switch boxes of flexibility F_s (planar, subset, or Wilton), and wire segments of several lengths. Routing is NP-complete and architecture dependent, and the field's algorithms form a tutorial in cost design. *Maze routing* expands a wavefront from the source and rip-ups and re-routes on failure. $A*$ interpolates between breadth-first and depth-first search with a single parameter,

$$f_i = (1-\alpha)\,(f_{i-1} + c_i) + \alpha\, d_i, \tag{12}$$

where c_i is the node cost and d_i the estimated cost to the target; domain negotiation adds a rank term r_d to steer traffic toward under-used track domains. *Pathfinder* [8] routes every net allowing resource overuse and iteratively raises the cost of overused and historically used resources,

$$f_i = (1 + h_n h_{\text{fac}})(1 + p_n p_{\text{fac}}) + b_{n,n+1}, \tag{13}$$

until no overuse remains. *VPR* [9] refines Pathfinder into routability-driven and timing-driven flavors (the latter adds an Elmore term and routes critical nets first) and modifies the wavefront to re-enter reached terminals at zero cost. *ROAD* replaces rip-up-and-reroute with a *bump-and-refit* paradigm whose learning-based and clique-based pruning speed the base search by 604×. *SAT-based* routing encodes connectivity and exclusivity as CNF clauses, so satisfiability is exactly routability—at the price of speed and memory. Table III summarizes the trade-offs: on minimum track count, VPR routability equals ROAD, VPR timing needs about 5% more, and SAT about 25% more; on runtime the timing-driven variant is fastest and SAT slowest.

VII. Clock Tree Synthesis and Timing Closure

A. Deferred-Merge Embedding for Prescribed Skew

Zero-skew clock routing distributes the clock so that every sink receives it at the same time. The *prescribed-skew* problem is the generalization in which each sink has its own required arrival time—the setting that useful-skew optimization of Section VII-B actually needs. The Deferred-Merge Embedding (DME) algorithm [10] solves it in two phases. In the bottom-up *merge* phase every node is annotated with a *merging segment*: the set of all positions at which the parent can be placed while maintaining the prescribed skew between its two children. In the top-down *embed* phase an actual position is chosen from each merging segment—the point nearest the parent—which minimizes total wirelength. A merging segment is a Manhattan arc, the L_1 analogue of a disc, and three operations suffice: the minimum distance between two arcs, the merge of two arcs offset by a tapping distance, and the nearest point of an arc to a query. The construction is $O(n \log n)$ for n sinks: all three implementations find the partition midpoint by linear-time selection—median-of-medians in Python and `nth_element` / `select_nth_unstable` in the C++ and Rust ports—and the cost is dominated by the balanced bipartition that builds the topology; Fig. 9 and Fig. 10 show the result under each delay model.

Algorithm 3 Deferred-Merge Embedding with prescribed skew

Require: sinks with required arrival times, delay model
1: build the balanced merging tree (alternating x/y bipartition)
2: **bottom-up:** for each node, $d \leftarrow$ distance between child segments; (extend, raw) $\leftarrow$ Tapping(T_L, T_R, d); merge the child segments offset by extend
3: **top-down:** place each node at the point of its segment nearest its parent
4: **elongate** when raw < 0 or raw $> d$
5: **return** the embedded clock tree and its skew

The tapping point is where the delay model enters. For the *linear* model, delay $= k \cdot$ length, the fraction of the inter-segment distance assigned to the left branch is

$$\text{extend_left} = \text{round}\Big(\frac{(T_L - T_R)/k + d}{2}\Big), \tag{14}$$

while for the *Elmore* model, delay $= R\,(C/2 + C_{\text{load}})$, it is the solution of a linear equation in the branch resistance and capacitance. The computation is factored into a pure function returning both the clamped `extend_left` and the raw value, so the caller can detect *elongation*: when the raw value is negative the right branch is too short and must be lengthened, when it exceeds the separation the left branch must be; a `need_elongation` flag records the event. Making the function pure—rather than mutating the tree as it computes—is what makes the linear and Elmore branches testable against one another and portable across languages.

Table III: FPGA detailed routers: qualitative trade-offs.

router	paradigm	unroutability detection	track count
VPR (routability)	negotiated rip-up/reroute	heuristic	minimal
VPR (timing)	+ Elmore cost	heuristic	$\sim 5\%$ more
ROAD	bump-and-refit	none (adds a track)	minimal
SAT	CNF satisfiability	formal proof	$\sim 25\%$ more

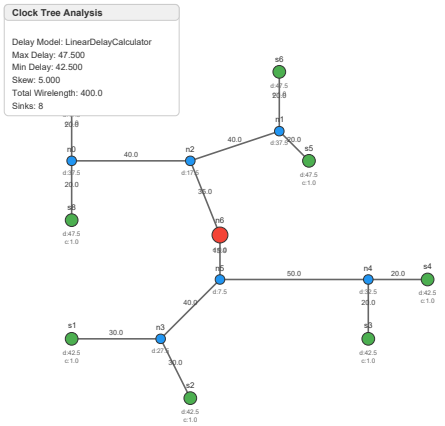


Figure 9: A prescribed-skew clock tree under the linear delay model; the panel reports maximum/minimum delay, skew and total wirelength.

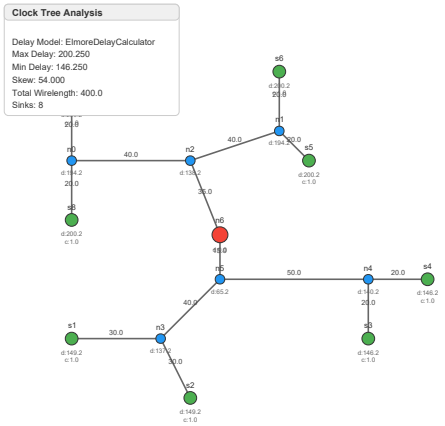


Figure 10: The same topology under the Elmore model: realistic RC delays with the prescribed skew preserved.

B. Static Timing, Useful Skew, and Delay Padding

Static timing analysis reduces timing verification to two linear inequalities per path. With T_{clk} the clock period, $T_{ck\to q}$ the clock-to-q delay, T_{logic} the combinational delay, and $T_{\text{skew}} = T_{ck2} - T_{ck1}$ the capture-minus-launch skew, the setup and hold constraints are

$$T_{\text{clk}} \geq T_{ck\to q} + T_{\text{logic}} + T_{\text{setup}} - T_{\text{skew}}, \qquad T_{ck\to q} + T_{\text{logic}} \geq T_{\text{hold}} + T_{\text{skew}}. \tag{15}$$

The *slack* of a path is the required time minus the arrival time; a design is closed when the worst negative slack (WNS) and total negative slack (TNS) vanish. The traditional view treats skew as an enemy to be zeroed, but the two inequalities show that a *positive* skew helps setup and hurts hold, and a negative skew the reverse. *Useful skew* exploits this: treating the clock arrival times s_i as variables, closure becomes a linear program

$$s_j - s_i \leq T_{\text{clk}} - D_{\max}(i,j), \qquad s_j - s_i \geq -D_{\min}(i,j), \tag{16}$$

whose objective minimizes TNS or maximizes yield. When hold violations remain after the easiest skew has been spent, *delay padding* inserts buffers or delay cells into short paths—greedy (fix the worst violator first), LP-based (optimal), or sensitivity-guided—trading power and area for hold margin. Late-stage fixes are delivered as engineering change orders: gate sizing and V_t swapping, buffer insertion or removal, layer assignment, and spare-cell utilization, in a pre-mask, post-mask, or buffer-only form. At advanced nodes deterministic STA is replaced by *statistical* STA, which treats delays as correlated random variables and replaces corner sign-off with a parametric yield.

VIII. Numerical Experiments

A. Cross-Language Verification

Every algorithm in this paper exists in three implementations—Python, C++20 and Rust—and the central experimental claim is not a speedup but an *equality*: the three ports agree on the reported quantities. The DME routine, for example, produces identical skew, wirelength and delays in all three languages on a battery of sink sets; the Steiner forest returns cost 17.0 with the same edge set on an 8×8 grid with five terminal pairs; and the rectilinear polygon routines agree on signed areas, with the Rust `signed_area` measurement (1.74 ns) below the C++ one (3.61 ns). Cross- language harnesses—Rust `cross_lang_verify.rs` (11 tests), the C++ `test_dme_algorithm.cpp` subcases, and matching assertions in the Python `test_dme_algorithm.py`—run in continuous integration. This equality is not free: it is where the interesting bugs live. Porting surfaced an elongation routine that mutated the tree before returning (fixed by returning a pure result), a triangular inverse that silently returned the reciprocal diagonal instead of the matrix inverse; each was caught only because a second implementation disagreed.

B. Removing Asymptotic Waste

The performance work in this paper is deliberately not about micro-optimization but about asymptotics, verified by a *differential oracle* that asserts the result is unchanged.

The netlist library's TSP 2-opt replaced an $O(n)$ rescore per candidate with a four-lookup delta identity,

$$\Delta = w(i-1, j-1) + w(i, j) - w(i-1, i) - w(j-1, j), \quad (17)$$

turning each sweep from $O(n^3)$ into $O(n^2)$ and its cycle-cover routine from $O(kE)$ into $O(E)$; the measured speedups were 99× (Python two-opt), 81–88× (Python cover), and about 10,000× (C++ cycle cover). The geometric toolkit, similarly, replaced an $O(n^2)$ tree-string construction with an $O(n)$ one (8.6× at $n = 1024$) and an $O(n^2)$ router-insertion scan with a nearest-point search ($\approx 4\times$). In each case a differential oracle over the Steiner, DME and routing routines confirmed zero mismatches. Table IV collects the representative gains.

C. Test Discipline

The three implementations carry 597 Python tests, 154 C++ test cases (766 assertions), and 373 Rust tests, plus differential and cross-language harnesses. The release gates are `pytest --cov`, `xmake test`, and `cargo test` respectively; a change is accepted only if all three are green and the differential oracle reports zero mismatches. This is the practical form of the exactness claim: the same algorithm on three runtimes, checked against itself.

IX. Discussion, Scope, and Threats to Validity

A. Validation Is Algorithmic, Not Silicon

The experiments in this paper establish *internal* properties—that DME reaches the prescribed skew, that the Steiner forest meets its 2-approximation, that the three language ports agree bit-for-bit—and they do so on *synthetic* data: Halton low-discrepancy site layouts and fixed-seed pseudo-random instances. No measured test-chip, foundry, or commercial netlist is used anywhere in the three implementations. Two consequences follow. First, the reported wirelength, delay and runtime figures are relative and machine-specific, not signoff predictions. Second, the algorithmic claims are of the kind that synthetic data *can* certify: a recovery problem with known ground truth, a matching among three implementations, and a feasibility property of an embedding. We therefore present the toolkit as a *research vehicle for geometry and algorithms*, not a signoff tool, and treat validation on measured silicon—which would add spatial non-stationarity, non-Gaussian process noise and irregular layout geometries—as the principal open task. The interchange readers (hMetis hypergraphs and Yosys JSON) live in the companion `netlistx` project rather than in the geometry toolkit, and the largest instances exercised are modest: a thousand-vertex rectilinear polygon, a few hundred sinks, and grids of a few thousand nodes.

B. Exact Complexity and Scalability Limits

Table V states the cost of each routine *as implemented*. The bounds are deliberately conservative: the primal-dual Steiner forest is $O(V^2)$ on a grid of V nodes, convex decomposition is $O(n^2)$ in the worst case ($O(n \log n)$ when the cuts are balanced), the global router is $O(n^2 k)$ for n terminals and k keepouts, and the congestion map is a rendering pass with no congestion computation at all. The DME construction is $O(n \log n)$ in all three implementations, each finding the partition midpoint by linear-time selection (median-of-medians in Python, `nth_element` / `select_nth_unstable` in the C++ and Rust ports); an earlier Python reference sorted each partition and was therefore $O(n \log^2 n)$, which a later output-preserving pass corrected. Scaling to a full-chip instance with $n > 10^3$ sites or sinks remains open. The geometric routines are protected by the domain rule that a polygon has at most about a hundred vertices; the network routines are not, and restoring scalability there (locally supported basis matrices, representative subsets, or blocked parallelism) is future work.

C. Physical Fidelity: Two First-Order Delay Models

The clock router implements exactly two delay models: the length-proportional `LinearDelayCalculator` and the lumped, single-pole `ElmoreDelayCalculator`, $D = R_{\text{wire}}(C_{\text{wire}}/2 + C_{\text{load}})$. Both are first-order; neither captures higher-order RC moments, crosstalk, or process variation. The toolkit likewise contains no statistical timer, no Liberty/CCS timing-table reader and no lithography model: the static-timing, useful-skew and delay-padding discussion of Section VII-B describes the surrounding *methodology*, not code in this repository. We treat this as a deliberate scope boundary—the contribution is exact geometry and combinatorial algorithms, and the delay model enters through a single `DelayCalculator` strategy so that a more faithful model can replace it without touching the DME core—but it means the toolkit cannot by itself sign off timing. Closing the gap would require at least a higher-order interconnect model, a crosstalk-aware stage-delay model [11], and a variability-aware timer in the spirit of block-based SSTA [12], [13]. Non-rectilinear and lithography-aware constraints are similarly out of scope: the geometry is rectilinear with integer 45-degree abstract lines, and design-rule, optical-proximity and multi-patterning constraints [14], [15], [16] are not modeled.

D. Flow Coverage: Global Routing, Not Signoff Routing

The router developed here is a *global* router: it produces Steiner-point-guided trees, avoids rectangular keepouts, supports a 3D nesting of the point type and can export a congestion map. It does **not** perform exact track assignment, detailed routing, via minimization, design-rule checking, pin-connection legalization or rip-up-and-reroute—none of these exists in the code—and its congestion map is a visualization of a caller-supplied grid, not an overflow or capacity model. Detailed routing for advanced nodes is a discipline of its own [17], [18], and we position the global router as the coarse stage that feeds such a tool. The FPGA-routing discussion of Section VI is correspondingly a *survey* of the literature [19], [20]

Table IV: Representative output-preserving speedups (measured, bit-identical results).

component	change	language	speedup
TSP 2-opt	$O(n^3) \to O(n^2)$ per sweep	Python	99×
cycle cover	$O(kE) \to O(E)$	C++	$\sim 10^4\times$
tree string	$O(n^2) \to O(n)$	C++	8.6×
router insertion	nearest-point search	Python	4×

Table V: Asymptotic cost of the principal routines, as implemented.

routine	time	note
primal-dual Steiner forest	$O(V^2)$	V = grid nodes; two edge scans per added edge
convex decomposition	$O(n^2)$ worst, $O(n \log n)$ balanced	one $O(n)$ scan per cut
DME build	$O(n \log n)$	linear-time median selection in all three implementations
global router	$O(n^2 k)$	n terminals, k keepouts
congestion map	$O(RC)$	rendering only
monotone hull / point-in-polygon	$O(n)$	rectilinear convex hull and ray casting

rather than an implementation; only the ASIC-oriented geometry and Steiner-forest routines are delivered in all three languages.

E. The Cost and Value of Cross-Language Parity

Maintaining three ports exacts a real price: roughly six hundred Python tests, over a hundred and fifty C++ test cases, and over three hundred Rust tests, each gated by its own CI job, plus cross-language harnesses that pin shared oracle constants. The price buys two things. First, it catches the class of bug a single implementation hides: an elongation routine that mutated the tree before returning, a tie-break that depended on set-iteration order, and an unsigned underflow in a Rust partition all surfaced only because a second implementation disagreed. Second, it constrains optimization to *output-preserving* changes, the discipline that let us remove the $O(n^3)$, $O(n^2)$ and $O(kE)$ hot paths while proving the results unchanged—and later to replace the Python $O(n \log^2 n)$ clock-tree bipartition with linear-time selection and to prune the Steiner-forest edge scans and the global router's insertion search. The constraint is not a bar on language-specific engineering: the ports use Rust `unsafe` and `no_std`, C++ `constexpr`/concepts and link-time optimization, and Python `__slots__`, and they differ in data layout wherever the result is invariant. What is forbidden is a fast change that moves the answer—precisely the property a differential oracle enforces.

F. Summary of Scope

In one sentence, this is an exact, cross-language geometry-and-algorithms toolkit for the front end of physical design, whose value is that its claims are reproducible and its failure modes are visible. The open problems it leaves are stated above: measured-silicon validation, sub-quadratic full-chip scaling, and advanced delay and lithography modeling.

X. Conclusion

We have developed the algorithmic foundations of a physical-design flow— rectilinear geometry, netlist optimization, Steiner-forest routing, global and FPGA routing, clock-tree synthesis and timing closure—and reported an exact, cross-language implementation of all of it. Three ideas recur. First, the domain's constraints (integer coordinates, rectilinearity, extreme scale but small objects) make simple, exact algorithms both possible and preferable; the pruning, tie-breaking and index arithmetic that follow from them are where the engineering lives. Second, the primal-dual schema unifies problems that look unrelated—graph covering and matching, and Steiner forest—and gives constant-factor guarantees that the experiments show are far from pessimistic in practice. Third, a trustworthy tool must be *verifiable*: it must be checked across implementations and against a differential oracle rather than trusted on its exit code. We validated the flow on synthetic grids, verified that the Python, C++ and Rust ports agree exactly, and documented the silent failure mode—asymptotically quadratic hot paths hidden behind a correct answer—that a research-grade toolkit must expose.

XI. Code Availability

The reference implementations, together with the experiments, cross-language harnesses and the differential oracles, are available as the `physdes` and `netlistx` projects, with Python, C++ and Rust variants, under the `luk036` organization (https://github.com/luk036).

[1] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA: Kluwer Academic Publishers, 1999.

[2] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht: Springer, 2011.

[3] S. M. Sait and H. Youssef, *VLSI physical design automation: Theory and practice*. Singapore: World Scientific, 1999.

[4] D.-T. Lee and C.-K. Wong, "Voronoi diagrams in L_1 (L_∞) metrics with two-dimensional storage applications," *SIAM J. Comput.*, vol. 9, no. 1, pp. 200–211, 1980.

[5] H. Edelsbrunner and E. P. Mücke, "Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms," *ACM Trans. Graph.*, vol. 9, no. 1, pp. 66–104, 1990.

[6] A. Agrawal, P. Klein, and R. Ravi, "When trees collide: An approximation algorithm for the generalized Steiner problem on networks," *SIAM J. Comput.*, vol. 24, no. 3, pp. 440–456, 1995.

[7] A. Ahmadi *et al.*, "A deterministic $(2 - 10^{-11})$-approximation for the Steiner forest problem." 2025.

[8] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in *Proc. ACM int. Symp. Field-programmable gate arrays (FPGA)*, 1995, pp. 111–117.

[9] V. Betz and J. Rose, "VPR: A new packing, placement and routing tool for FPGA research," in *Proc. Int. Workshop on field-programmable logic and applications (FPL)*, 1997, pp. 213–222.

[10] T.-H. Chao, Y.-C. Hsu, J.-M. Ho, K. D. Boese, and A. B. Kahng, "Zero skew clock routing with minimum wirelength," *IEEE Trans. Circuits Syst. II: Analog Digit. Signal Process.*, vol. 39, no. 11, pp. 799–814, 1992.

[11] K. L. Shepard, "Design methodologies for noise in digital integrated circuits," in *Proc. 35th annual design automation conf. (DAC)*, 1998, pp. 94–99. doi: 10.1145/277044.277062.

[12] C. Visweswariah, K. Ravindran, K. Kalafala, S. G. Walker, and S. Narayan, "First-order incremental block-based statistical timing analysis," in *Proc. 41st annual design automation conf. (DAC)*, 2004, pp. 331–336. doi: 10.1145/996566.996663.

[13] S. S. Sapatnekar, *Timing*. Boston, MA: Springer, 2004. doi: 10.1007/b117318.

[14] L. W. Liebmann, "Layout impact of resolution enhancement techniques: Impediment or opportunity?" in *Proc. Int. Symp. Physical design (ISPD)*, 2003, pp. 110–117. doi: 10.1145/640000.640026.

[15] A. B. Kahng, C.-H. Park, X. Xu, and H. Yao, "Layout decomposition for double patterning lithography," in *Proc. IEEE/ACM int. Conf. Computer-aided design (ICCAD)*, 2008, pp. 465–472. doi: 10.1109/ICCAD.2008.4681616.

[16] J.-R. Gao and D. Z. Pan, "Flexible self-aligned double patterning aware detailed routing with prescribed layout planning," in *Proc. Int. Symp. Physical design (ISPD)*, 2012, pp. 25–32.

[17] A. B. Kahng, L. Wang, and B. Xu, "TritonRoute: The open-source detailed router," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 3, pp. 547–559, 2021, doi: 10.1109/TCAD.2020.3003234.

[18] A. Hashimoto and J. Stevens, "Wire routing by optimizing channel assignment within large apertures," in *Proc. 8th design automation workshop (DAC)*, 1971, pp. 155–169.

[19] S. D. Brown, J. Rose, and Z. G. Vranesic, "A detailed router for field-programmable gate arrays," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 5, pp. 620–628, 1992, doi: 10.1109/43.127623.

[20] G. G. F. Lemieux and S. D. Brown, "A detailed routing algorithm for allocating wire segments in field-programmable gate arrays," in *Proc. ACM/SIGDA physical design workshop*, Lake Arrowhead, CA, 1993, pp. 215–226.